

Technical Assessment - On-Device NPU Deployment

Role: AI/ML Engineer (Edge Deployment)

Format: Take-home technical assessment

1. Company Context

KawachX is an edge-deployed computer vision system for industrial safety monitoring - fire, smoke, and person detection running entirely on-device, with no dependency on a cloud connection for inference. The target hardware is a **Qualcomm QCS6490** SoC (Hexagon NPU), integrated into a purpose-built edge appliance.

2. Current State

- A 3-class object detection model (**Fire, Smoke, Person**) has been trained and exported to **ONNX (FP32)**. The model is architecturally a **YOLOv8**-style detector.
- The FP32 model has been validated for correctness on CPU (ONNX Runtime) against real test imagery and performs as expected.
- File: `kavachx_testing/models/new_3class_best_FP32.onnx`

3. Objective

Deploy this model to run **inference on the NPU**, not the CPU or GPU.

This is a hard requirement, not a preference. Edge inference on the NPU is necessary for:

- **Power efficiency** - this is a 24/7, always-on deployment; CPU-bound inference is not viable at the target duty cycle and thermal envelope.

- **Latency and throughput** - the NPU is purpose-built tensor-acceleration hardware; CPU inference cannot meet the real-time, multi-camera throughput target on this SoC.
- **Headroom** - the CPU needs to remain available for camera ingestion, decoding, alerting, and other pipeline responsibilities running concurrently on the same device.

4. The Core Technical Problem

Qualcomm's NPU runtime requires models to be compiled into a **quantized INT8 QNN context binary** (a `.bin` file) - the NPU cannot execute the FP32 model directly.

Producing a correct, working `.bin` from the FP32 source model has proven to be the central technical challenge of this deployment. This is the problem you are being asked to solve.

5. What You're Given

- **The source model:** `kavachx_testing/models/new_3class_best_FP32.onnx`
- `npu_worker/`: a C++ runtime that loads a compiled QNN context binary and serves inference requests against the Hexagon NPU via the QNN C API. This is the harness your compiled model needs to run inside - review its expectations (input/output tensor shapes, dtypes, and the IPC contract it exposes) as part of your approach.
- Two prior `.bin` artifacts are also present in `kavachx_testing/models/` from previous internal attempts. They are provided for reference only - **do not assume either is correct or usable as a starting point**. Verifying their status is part of the assessment.
- Qualcomm's QAI RT SDK is available on the target device, and Qualcomm AI Hub (cloud-based compile/quantize/link service) is an option if you choose to use it - set up your own account if so.

6. The Task

1. Produce a quantized INT8 `.bin` file from the source FP32 model that runs correctly on the NPU via `npu_worker`.
2. Demonstrate it working end-to-end: load the model into `npu_worker`, run inference against real image input, and show the output is numerically valid and consistent with the FP32 model's own predictions on the same input.
3. Document your approach: what you tried, what worked, what didn't, and why.

7. You Are Not Required to Accept the Current Approach

You are explicitly encouraged to exercise engineering judgment beyond "make the given model quantize successfully." In particular, you may:

- Propose a **different base architecture** (YOLOv4, YOLOv5, YOLOv6, or another detector family) if you believe it is better suited to this NPU/quantization target, and justify why.
- Critique or challenge the **existing `npu_worker` design** - its IPC protocol, its assumptions about input/output tensor format, or its overall architecture - if you believe it is a poor fit for a correctly quantized model, and propose an alternative.
- Propose a materially different deployment approach altogether, provided you can justify the tradeoffs.

We are evaluating engineering judgment as much as raw execution. If you conclude the current approach is wrong, say so and explain why - that is a valid and valuable outcome of this assessment.

8. Deliverables

- The final, working `.bin` file (or your best result, if full success wasn't reached - partial, well-documented progress is far more valuable than an unexplained final artifact).
- A short written report covering:

- Your approach and the reasoning behind key decisions
- Evidence of correctness (not just "it loads" - show real output validation against the FP32 reference)
- Any architectural concerns or recommendations regarding the current model choice or `npu_worker` design
- Known limitations or unresolved issues, if any

9. Evaluation Criteria

- Technical correctness of the final result
- Depth and rigor of validation (how you proved correctness, not just claimed it)
- Quality of engineering judgment - problem diagnosis, tradeoff reasoning, and willingness to challenge assumptions where warranted
- Clarity of written communication in the final report

Questions on scope or ambiguous requirements are expected and welcome - raise them rather than guessing silently on anything that materially changes your approach.